

Docker Demo 4

Docker container as inference service

1 Project Setup

First you create a dedicated project directory and set up a Python environment.

Directory Structure

You start by creating a directory such as:

```
docker-demo-4/
```

You then make this the current working directory:

```
mkdir docker-demo-4
cd docker-demo-4
```

Make sure you are in the right python environment.

Dependencies

You will probably need to install `uvicorn`, `fastapi`, `pydantic`. Use `pip` to install these accordingly.

You will probably need the following:

```
# requirements.txt
mlflow==2.9.1
scikit-learn==1.4.2
numpy==1.26.4
fastapi==0.110.0
uvicorn==0.29.0
pydantic==2.6.4
```

Make sure to install these dependencies.

2 Training and Registering the Breast Cancer Classifier

In this section you train a model on the Breast Cancer Wisconsin dataset and log it to MLflow using a file-based tracking URI located in a subdirectory called `mlruns` in your project.

Training Script with MLflow Logging

You create a file called `train.py`:

```

import mlflow
import mlflow.sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

mlflow.set_experiment("breast_cancer_experiment")

def main():
    data = load_breast_cancer()
    X = data.data
    y = data.target
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        random_state=42)
    with mlflow.start_run() as run:
        model = RandomForestClassifier(
            n_estimators=100,
            random_state=42
        )
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        acc = accuracy_score(y_test, preds)
        print(f"Accuracy: {acc:.4f}")
        # Log test accuracy metric
        mlflow.log_metric("accuracy", acc)
        # Log and register model in the MLflow Model Registry
        mlflow.sklearn.log_model(
            model,
            artifact_path="model",
            registered_model_name="BreastCancerClassifier"
        )
        print(f"Run ID: {run.info.run_id}")

if __name__ == "__main__":
    main()

```

Now you run the training script:

```
python train.py
```

This command creates a directory called `mlruns` inside your project. That folder now contains the experiment metadata, run information, and the registered model `BreastCancerClassifier` along with its first version.

3 Assigning the **champion** Alias Using the MLflow UI

Next you use the MLflow UI to inspect runs and assign the alias `champion` to the best version of `BreastCancerClassifier`.

Launching the MLflow UI

You start the MLflow UI by pointing it at the same file-based store you used in `train.py`:

```
mlflow ui
```

Now you open your browser and navigate to:

```
http://localhost:5000
```

or

```
http://127.0.0.1:5000
```

You should see your experiment `breast_cancer_experiment` and its runs.

Assigning the **champion** Alias via UI

In the UI you now perform the following steps:

1. Go to the **Models** tab.
2. Click on `BreastCancerClassifier`.
3. Select the version you want to mark as the current best model (for example version 1).
4. Add a new alias named `champion` and assign it to that version.

After you complete this step the registry will contain an alias mapping such that:

```
models:/BreastCancerClassifier@champion
```

always points to the current best model version you selected.

You can now stop the MLflow UI if you want; the alias and registry metadata are stored in the `mlruns` folder.

4 Building the FastAPI Inference Service (**app.py**)

Now you create an inference API that loads the aliased model and exposes it as a web service. We will export the `model.pkl` file and use it in the inference time.

Inference Application Code

First, let's export the `model.pkl` file from mlflow:

```
# export_model_to_pkl.py
import os
import mlflow
import mlflow.sklearn
import joblib

MODEL_NAME = "BreastCancerClassifier"
MODEL_ALIAS = "champion"

model_uri = f"models:{MODEL_NAME}@{MODEL_ALIAS}"
print(f>Loading model from MLflow URI: {model_uri}")

sk_model = mlflow.sklearn.load_model(model_uri)
print("Loaded model:", type(sk_model))

# # Save as plain pickle using joblib
joblib.dump(sk_model, "model.pkl")
print("Saved model.pkl in current directory")
```

Now do inference with this `.pkl` model file:

```

import numpy as np
import joblib
from fastapi import FastAPI
from pydantic import BaseModel
import warnings
warnings.filterwarnings("ignore")

app = FastAPI()

# Load model.pkl at startup
print("Loading model from model.pkl...")
model = joblib.load("model.pkl")
print("Model loaded:", type(model))

class PredictionInput(BaseModel):
    features: list[float]

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(input_data: PredictionInput):
    if len(input_data.features) != 30:
        return {"error": "Expected 30 features"}
    arr = np.array(input_data.features).reshape(1, -1)
    prediction = model.predict(arr)
    return {
        "prediction": int(prediction[0])
    }

```

At this point you can test the app locally (outside Docker) by running:

```
uvicorn app:app --reload --port 8000
```

Then make a prediction request by

```

curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [1,2,3,4,5,6,7,8,9,1,2,3,4,5,6,7,8,9,1,2,3,4,5,6,7,8,9,1,2,3]}'

```

You should see a prediction result. Now, we will move to containerization of this app.

5 Preparing Docker Artifacts

Now you containerize the inference application. The container will not include MLflow artifacts; instead it will access the host's mlruns directory via a volume mount.

Dockerfile

You create a file named Dockerfile in the project root:

```

FROM python:3.11-slim
WORKDIR /app

```

```
# Install system-level build tools (often needed by ML libs)
RUN apt-get update && apt-get install -y \
build-essential \
&& rm -rf /var/lib/apt/lists/*
# Install Python dependencies
COPY requirements.txt .
RUN pip install --upgrade pip && \
pip install --no-cache-dir -r requirements.txt
# Copy app code and the exported model artifact
COPY app.py .
COPY model.pkl .
# Expose FastAPI port
EXPOSE 8000
# Run FastAPI with uvicorn in the container
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

.dockerignore

To keep your image small and to avoid accidentally loading data or artifacts into the build context, you create a `.dockerignore` file:

```
# .dockerignore

__pycache__/*
*.pyc
*.pyo
*.pyd

.venv/
venv/

.git/
.gitignore

# MLflow artifacts and DB (we're only shipping model.pkl)
mlruns/
mlflow.db

data/
datasets/

*.ipynb
*.log

.DS_Store
```

This ensures that when you build the image, the `mlruns` directory and other large or irrelevant files are not sent to the Docker daemon.

Building the Docker Image

You now build the Docker image from the project root:

```
docker build -t bc-inference:1.0 .
```

This command creates an image named `bc-inference` with tag `1.0`. The image contains Python, the dependencies from `requirements.txt`, and the `app.py` FastAPI service and also the model artifact.

6 Running the Container with inference service

```
docker run -p 8000:8000 bc-inference:1.0
```

Testing the Containerized Inference API

You can now send requests from your host machine to the service running inside the container.

For a simple health check you navigate to:

```
http://localhost:8000/health
```

You should see a JSON response:

```
{"status": "ok"}
```

You then send a prediction request, for example:

```
curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{"features": [14.0, 20.0, 90.0, 600.0, 0.10, 0.20, 0.30, 0.10, 0.20, 0.06,
                  0.40, 1.20, 2.30, 40.0, 0.005, 0.020, 0.030, 0.010, 0.020, 0.003,
                  15.0, 25.0, 100.0, 700.0, 0.15, 0.25, 0.35, 0.15, 0.30, 0.07]}'
```

If everything is correctly wired, the container will return a JSON object containing the predicted class.

In class practice

Finish everything above. And then, find a way that will check automatically the `model.pkl` file and rebuilds and runs the docker container automatically as soon as a new model champion comes. Automate this. So that: when you run `train.py` with a new ML model and make it a new champion in MLFlow, docker container automatically gets that new model and does inference with it.